

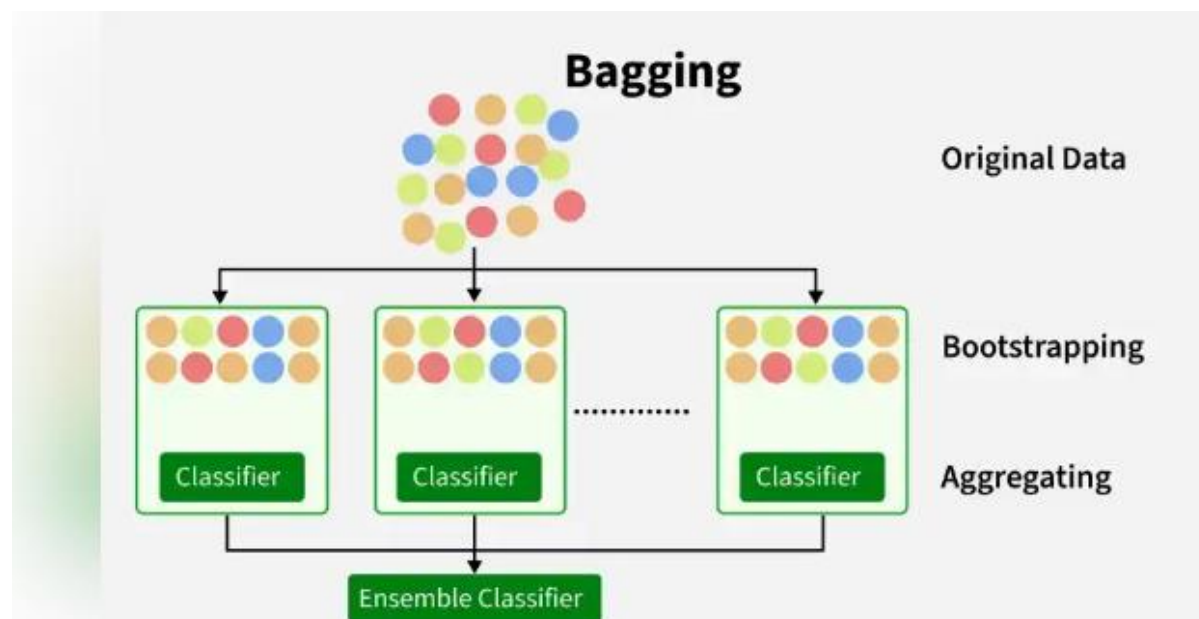
1 Why does bagging reduce variance but not bias?

Bagging and Boosting are both ensemble learning techniques used to improve model performance by combining multiple models. The main difference is that:

- **Bagging** reduces variance by training models independently.

It aims to reduce model variance by training multiple models on different random subsets of the dataset. These subsets are created using bootstrapping, where data points are sampled with replacement.

Each model is trained independently, and their predictions are combined using voting for classification or averaging for regression. As models are trained independently, Bagging works well with high-variance models like decision trees. **Bagging reduces variance by averaging unstable models but does not reduce bias because it does not change the underlying model assumptions.**



- **Boosting** reduces bias by training models sequentially, focusing on previous errors.

Boosting focuses on improving model accuracy by training models sequentially. Each new model pays more attention to the data points that were misclassified by previous models. Over time, the ensemble becomes better at handling difficult cases.



2 Prove that averaging reduces variance.

Let

$$X_1, X_2, \dots, X_n$$

be n random variables representing predictions from different models, each with mean μ and variance σ^2 . Let the pairwise correlation between any two estimators be ρ .

Define the averaged estimator:

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

The variance of the average is:

$$\text{Var}(\bar{X}) = \text{Var}\left(\frac{1}{n} \sum_{i=1}^n X_i\right) = \frac{1}{n^2} \text{Var}\left(\sum_{i=1}^n X_i\right)$$

Using the variance expansion for correlated variables:

$$\text{Var}\left(\sum X_i\right) = \sum \text{Var}(X_i) + 2 \sum_{i < j} \text{Cov}(X_i, X_j)$$

Substituting $\text{Var}(X_i) = \sigma^2$ and $\text{Cov}(X_i, X_j) = \rho\sigma^2$, we obtain:

$$\text{Var}\left(\sum X_i\right) = n\sigma^2 + n(n-1)\rho\sigma^2$$

Therefore,

$$\text{Var}(\bar{X}) = \frac{1}{n^2} [n\sigma^2 + n(n-1)\rho\sigma^2] = \sigma^2 \left(\frac{1}{n} + \frac{n-1}{n} \rho \right)$$

If the estimators are independent, then $\rho = 0$, giving:

$$\text{Var}(\bar{X}) = \frac{\sigma^2}{n}$$

Since $\frac{\sigma^2}{n} < \sigma^2$ for $n > 1$, the averaged estimator has lower variance than any single estimator.

The variance of a single model is σ^2 , whereas the variance of the averaged model decreases to σ^2/n , showing that averaging n models reduces variance.

Averaging works because random errors from individual models tend to cancel each other out. The variance reduction becomes stronger when:

- the number of models increases
- the models are less correlated

This explains why ensemble methods like bagging and Random Forest produce more stable predictions than a single model.

3 Derive out-of-bag error estimator.

The Out-of-Bag (OOB) error is a technique used to estimate the prediction error of machine learning models that employ bootstrap aggregating (bagging), such as random forests and bagged decision trees. It evaluates model performance using the training samples that were not included in the bootstrap sample for each tree. Because these observations are effectively unseen during the training of those trees, the OOB error provides an accurate and unbiased estimate of the model's generalization performance without requiring a separate validation set or cross-validation.

Derivation of the OOB Error Estimator

Consider a training dataset

$$D = \{(x_i, y_i)\}_{i=1}^n.$$

In bagging, we generate B bootstrap samples from the training data (sampling with replacement). For each bootstrap sample b , a tree T_b is trained.

For any observation i , the probability that it is not selected in one bootstrap sample of size n is

$$\left(1 - \frac{1}{n}\right)^n \approx e^{-1} \approx 0.368.$$

Thus, roughly 36.8% of the data are out-of-bag for any given tree.

Let B_i denote the set of trees for which observation i was out-of-bag.

The OOB prediction for observation i is computed using only those trees.

For regression:

$$\hat{y}_i^{\text{OOB}} = \frac{1}{|B_i|} \sum_{b \in B_i} T_b(x_i).$$

For classification (majority vote):

$$\hat{y}_i^{\text{OOB}} = \text{mode}\{T_b(x_i) : b \in B_i\}.$$

The overall OOB error is then computed as follows.

For regression (Mean Squared Error):

$$\text{OOB Error} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i^{\text{OOB}})^2.$$

For classification (misclassification rate):

$$\text{OOB Error} = \frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{y}_i^{\text{OOB}}),$$

where $I(\cdot)$ is the indicator function.

Because each observation is predicted only by trees that did not use it during training, the OOB error provides an approximately unbiased estimate of the model's test error and helps detect overfitting while avoiding the need for cross-validation.

4 Why does feature randomness decorrelate trees?

Feature randomness, also known as feature bagging or “the random subspace method”, generates a random subset of features, which ensures low correlation among decision trees.

- In a normal Decision Tree, all features are considered at every split, so a very strong feature is likely to be selected at the top of every tree.
- Even when bootstrap sampling is used, the same strong feature often dominates the splits in different trees.
- As a result, the trees grow in a very similar structure.
- Similar tree structures lead to similar predictions.
- Similar predictions mean the trees are highly correlated.
- In a Random Forest, only a random subset of features is considered at each split.
- The strong feature may not be available in some splits.
- This forces the tree to use other features for splitting.
- Different trees therefore learn different structures.
- Because the structures differ, the trees make different errors.
- When trees make different errors, their predictions are less correlated.

Feature randomness decorrelates trees because it introduces diversity in tree construction, preventing the same dominant feature from controlling every tree and reducing similarity among trees.

5 Compare Random Forest vs Gradient Boosting mathematically.

Random Forests:

- Random Forests construct multiple decision trees independently.

- Each tree is built on a randomly selected subset of the training data (bootstrapping) and a random subset of features.
- The predictions from all trees are then averaged (for regression) or majority voted (for classification) to obtain the final prediction.
- This parallel approach makes Random Forests less prone to overfitting and more robust, as each tree learns from different subsets of the data.

Let

- $T_1, T_2, \dots, T_n$ be the individual trees
- x be an input data point

The final prediction for **classification** is:

$$\hat{y} = \text{mode}(T_1(x), T_2(x), \dots, T_n(x))$$

The model chooses the most common class predicted by the trees.

For **regression**, the final prediction is:

$$\hat{y} = \frac{1}{n} \sum_{i=1}^n T_i(x)$$

The model averages the predictions of all trees.

Mathematical Nature of RF

- Trees are built **independently**.

- There is **no residual correction**.
- The final model is a **simple aggregation** (mode or mean).
- The objective is mainly **variance reduction** through averaging.

The model form is:

$$F(x) = \frac{1}{n} \sum_{i=1}^n T_i(x)$$

Gradient Boosting Trees (GBT):

- GBT builds decision trees sequentially.
- Each new tree in the ensemble focuses on reducing the errors made by the previous ones.
- The algorithm fits each new tree on the residual errors (the difference between the predicted and actual values) of the previous ensemble.
- This sequential nature allows GBT to learn complex relationships in the data but makes it more prone to overfitting, especially if not properly regularized.

Let

- $F(x)$ be the current model
- r_i be the residual for the i^{th} point

The residual is calculated as:

$$r_i = y_i - F(x_i)$$

In each iteration, a new tree $h(x)$ is trained to minimize the residuals.

The updated model becomes:

$$F(x) \leftarrow F(x) + \eta h(x)$$

Where:

- η is the learning rate
- $h(x)$ is the new tree trained on residuals

Mathematical Nature of GB

- Trees are built **sequentially**.
- Each tree corrects previous errors.
- Model is an **additive expansion**.
- It performs **stage-wise optimization**.

Final model after M iterations:

$$F_M(x) = \sum_{m=1}^M \eta h_m(x)$$

6 When does Random Forest fail?

- When data is extremely high-dimensional and sparse: Random feature selection may repeatedly pick irrelevant features.

- Random Forest predicts by averaging training outcomes. It cannot predict values beyond the range of training data :**Extrapolation** means making predictions **outside the range of the training data**.
- If the true relationship is strongly linear, RF may Use many splits unnecessarily and Require more data to approximate a simple linear function
- When Classes Are Highly Imbalanced, Unless class weights or balancing methods are applied.
- When Classes Are Highly Imbalanced, Unless class weights or balancing methods are applied.
- If dataset is extremely small: Trees become unstable and
- High variance may persist
- When features are highly correlated

7 Analyze computational complexity vs single tree.

Computational complexity measures how much **time** or **memory** an algorithm requires as the input size increases. The two main types are time complexity and space complexity.

In single decision tree,

Let:

- n = number of training samples
- p = number of features
- d = depth of tree

♦ Training Complexity

At each split:

- We evaluate all p features.
- For each feature, sorting costs $O(n \log n)$.

Overall training complexity:

$$O(p n \log n)$$

If tree grows deep (worst case):

$$O(p n^2)$$

♦ Prediction Complexity

For one sample:

- Traverse from root to leaf.

$$O(d)$$

If tree is balanced:

$$d \approx \log n$$

So prediction:

$$O(\log n)$$

8 Explain consistency of Random Forest.

Consistency means that an estimator becomes more accurate as the sample size increases.

Random Forest is consistent under certain theoretical conditions:

1. **Number of samples increases:** $n \rightarrow \infty$
2. **Number of trees increases:** $B \rightarrow \infty$
3. **Trees grow deep enough:** Leaf nodes must shrink as data increases.
4. **Splitting becomes more localized:** The diameter of terminal nodes must shrink to zero.

9 Why are forests robust to noise?

Random Forests are robust to noise because of the following reasons:

- **Bootstrap sampling (bagging):** Each tree is trained on a different random subset of the data. Noisy observations may not appear in every tree, reducing their overall impact.
- **Averaging predictions:** Final prediction is obtained by averaging (regression) or majority voting (classification). Random noise tends to cancel out when aggregated across many trees.
- **Variance reduction:** Averaging multiple trees reduces variance, which helps stabilize predictions even when data contains noise.
- **Feature randomness:** At each split, only a random subset of features is considered. Noisy or irrelevant features are less likely to dominate all trees.
- **Deep trees with independence:** Individual trees may overfit noisy samples, but since trees are built independently, their errors are not perfectly correlated, so ensemble averaging reduces their effect.

- **Law of Large Numbers effect:** As the number of trees increases, the impact of random fluctuations due to noise diminishes.

10 How do forests handle high-dimensional data?

- **Feature randomness:** At each split, only a random subset of features is considered instead of all features. This prevents the curse of dimensionality from dominating the model.
- **Automatic feature selection:** Important features are naturally chosen more often during splitting, while irrelevant features are rarely selected.
- **Reduced overfitting:** Since not all features are used at every split, the model avoids relying too heavily on noisy or irrelevant predictors.
- **Implicit dimensionality reduction:** Although all features exist in the dataset, each tree only uses a subset of them, effectively reducing dimensional complexity.
- **Handling many irrelevant features:** Even if many features are uninformative, averaging across trees ensures that useful signals dominate in final predictions.